Solution: 'K' Closest Numbers

Problem Statement

Given a sorted number array and two integers 'K' and 'X', find 'K' closest numbers to 'X' in the array. Return the numbers in the sorted order. 'X' is not necessarily present in the array.

Example 1:

```
Input: [5, 6, 7, 8, 9], K = 3, X = 7
Output: [6, 7, 8]
```

Example 2:

```
Input: [2, 4, 5, 6, 9], K = 3, X = 6
Output: [4, 5, 6]
```

Example 3:

```
Input: [2, 4, 5, 6, 9], K = 3, X = 10
Output: [5, 6, 9]
```

Solution

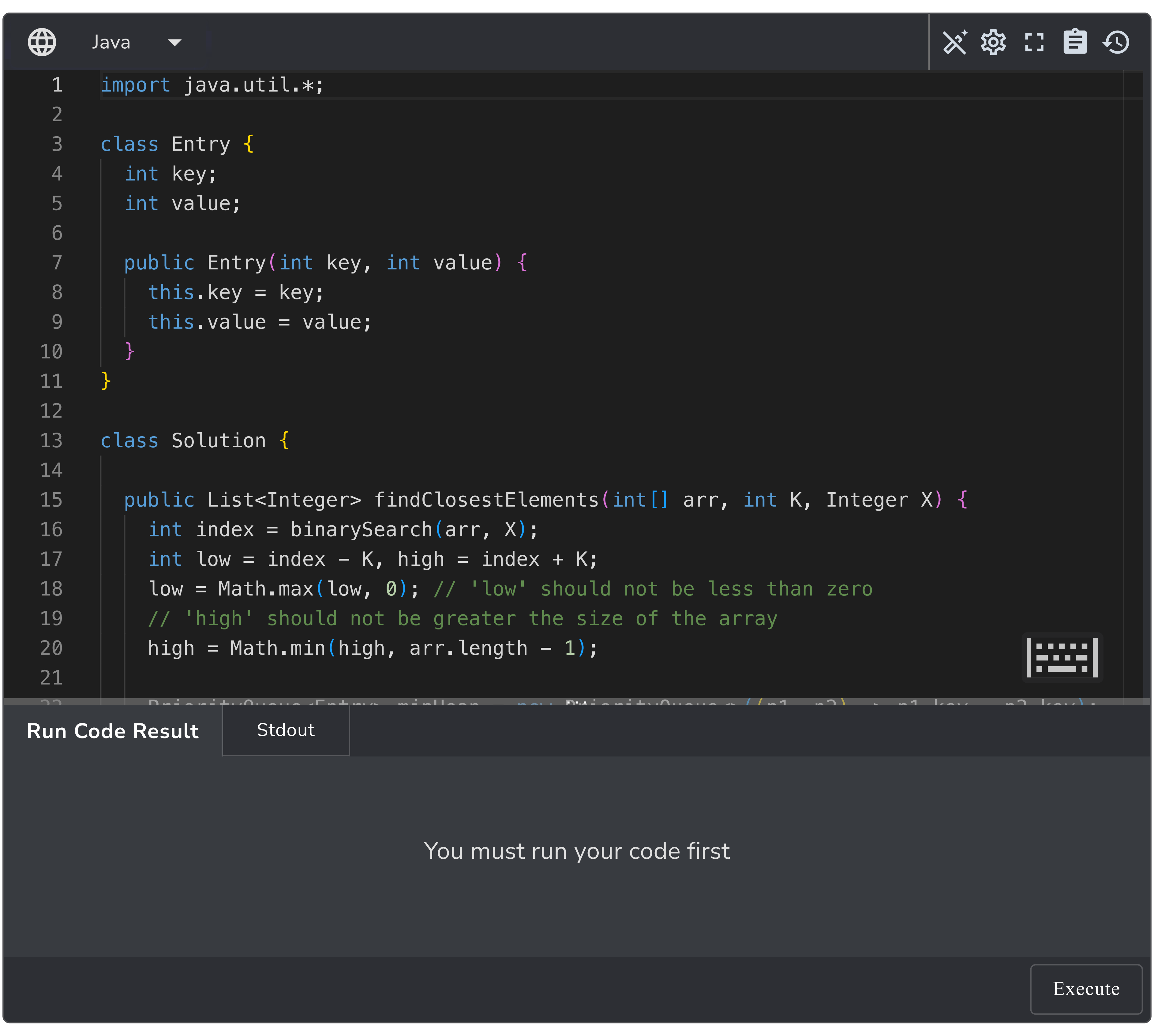
This problem follows the `Top 'K' Numbers` pattern. The biggest difference in this problem is that we need to find the closest (to 'X') numbers compared to finding the overall largest numbers. Another difference is that the given array is sorted.

Utilizing a similar approach, we can find the numbers closest to 'X' through the following algorithm:

1. Since the array is sorted, we can first find the number closest to 'X' through **Binary Search**. Let's say that number is 'Y'.
2. The 'K' closest numbers to 'Y' will be adjacent to 'Y' in the array. We can search in both directions of 'Y' to find the closest numbers.
3. We can use a heap to efficiently search for the closest numbers. We will take 'K' numbers in both directions of 'Y' and push them in a **Min Heap** sorted by their absolute difference from 'X'. This will ensure that the numbers with the smallest difference from 'X' (i.e., closest to 'X') can be extracted easily from the **Min Heap**.
4. Finally, we will extract the top 'K' numbers from the **Min Heap** to find the required numbers.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Entry {
  int key;
  int value;

  public Entry(int key, int value) {
    this.key = key;
    this.value = value;
  }
}

class Solution {

  public List<Integer> findClosestElements(int[] arr, int K, Integer X) {
    int index = binarySearch(arr, X);
    int low = index - K, high = index + K;
    low = Math.max(low, 0); // 'low' should not be less than zero
    // 'high' should not be greater the size of the array
    high = Math.min(high, arr.length - 1);

```

Time Complexity

The time complexity of the above algorithm is **O(logN+K∗logK)**. We need O(logN) for Binary Search and O(K∗logK) to insert the numbers in the **Min Heap**, as well as to sort the output array.

Space Complexity

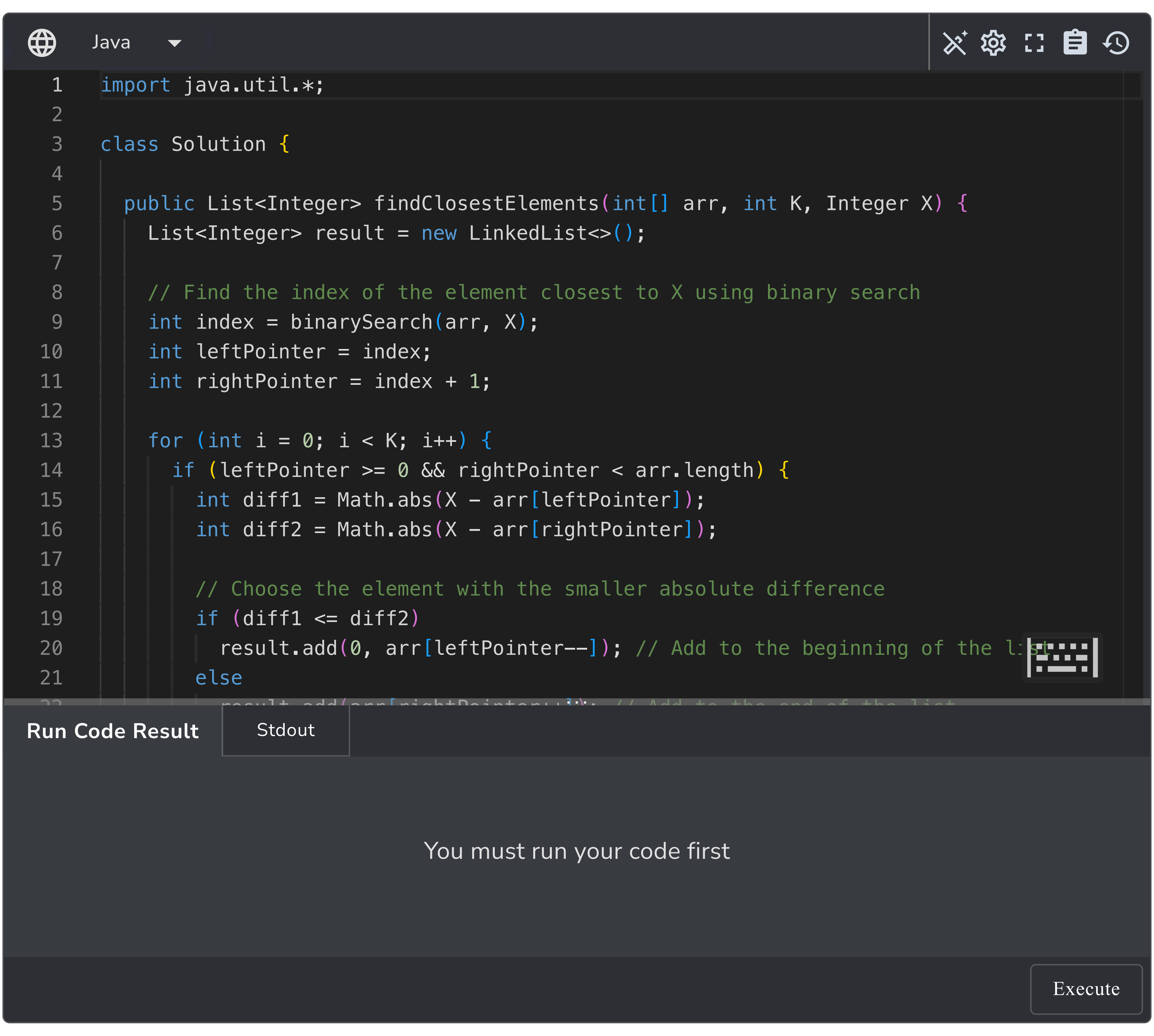
The space complexity will be O(K), as we need to put a maximum of 2K numbers in the heap.

Alternate Solution using Two Pointers

After finding the number closest to 'X' through **Binary Search**, we can use the **Two Pointers** approach to find the 'K' closest numbers. Let's say the closest number is 'Y'. We can have a `left` pointer to move back from 'Y' and a `right` pointer to move forward from 'Y'. At any stage, whichever number pointed out by the `left` or the `right` pointer gives the smaller difference from 'X' will be added to our result list.

To keep the resultant list sorted we can use a **Queue**. So whenever we take the number pointed out by the `left` pointer, we will append it at the beginning of the list and whenever we take the number pointed out by the `right` pointer we will append it at the end of the list.

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public List<Integer> findClosestElements(int[] arr, int K, Integer X) {
    List<Integer> result = new LinkedList<>();

    // Find the index of the element closest to X using binary search
    int index = binarySearch(arr, X);
    int leftPointer = index;
    int rightPointer = index + 1;

    for (int i = 0; i < K; i++) {
      if (leftPointer >= 0 && rightPointer < arr.length) {
        int diff1 = Math.abs(X - arr[leftPointer]);
        int diff2 = Math.abs(X - arr[rightPointer]);

        // Choose the element with the smaller absolute difference
        if (diff1 <= diff2)
          result.add(0, arr[leftPointer--]); // Add to the beginning of the list
        else
```

Time Complexity

The time complexity of the above algorithm is $O(logN + K)$. We need O(logN) for Binary Search and O(K) for finding the 'K' closest numbers using the two pointers.

Space Complexity

If we ignoring the space required for the output list, the algorithm runs in constant space O(1).

← Previous
'K' Closest Numbers (Medium)

Next →
Maximum Distinct Elements (Medium)

☐ Mark as Completed